

[< Back](#)

EMQX Performance Tuning: TCP SYN Queue and Accept Queue



EMQX Team

Dec 15, 2023

Product

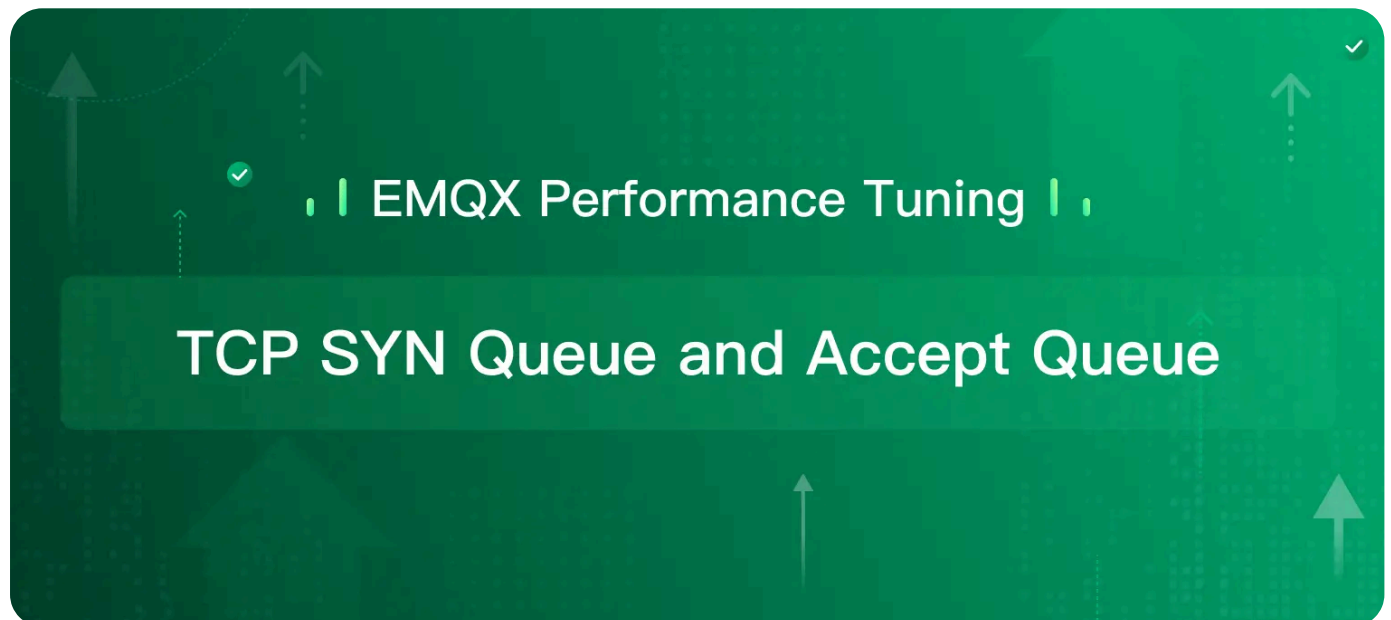


Table of Contents

- [SYN Queue and Accept Queue](#)
- [What Happens When the SYN Queue Overflows?](#)
- [What Happens When the Accept Queue Overflows?](#)
- [Increase the Size of the SYN and Accept Queues](#)
- [Conclusion](#)

In the previous blog ([EMQX Performance Tuning: Max Connections and File Descriptors](#)), we delved into the relationship between MQTT connections and file descriptors, and introduced how to modify kernel parameters related to file descriptors to break through the default maximum connection limit.

However, you may find that the client's connection request fails even if the total number of MQTT connections on the current server has not reached the file descriptor limit. You will see the following overflowed and SYN dropped counts increasing continuously when you run the following command:

```
$ watch -d 'netstat -s | grep -i "listen"'  
2091 times the listen queue of a socket overflowed  
3418 SYNs to LISTEN sockets dropped
```



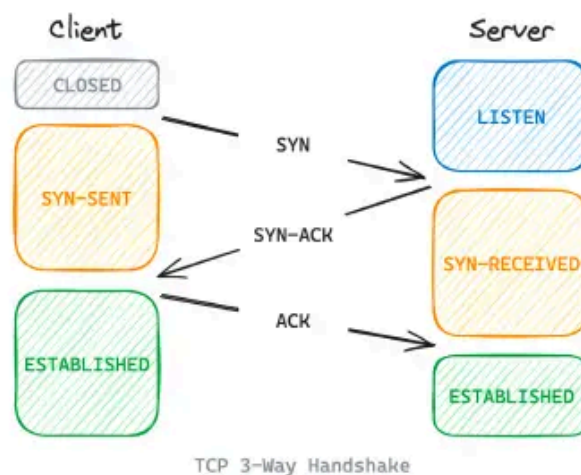
This article will introduce the reasons for this phenomenon and how to solve this problem through kernel parameter tuning.

SYN Queue and Accept Queue

The two counts mentioned in the previous section usually indicate that the SYN and accept queues have overflowed. Once overflow occurs, the normal connection of the client will be affected.

So first, we need to understand what the SYN and accept queues are.

As an application layer protocol built on top of TCP, MQTT connections always require a TCP connection to be established first, before we can send the CONNECT packet of the MQTT protocol. Establishing a TCP connection requires a three-way handshake:



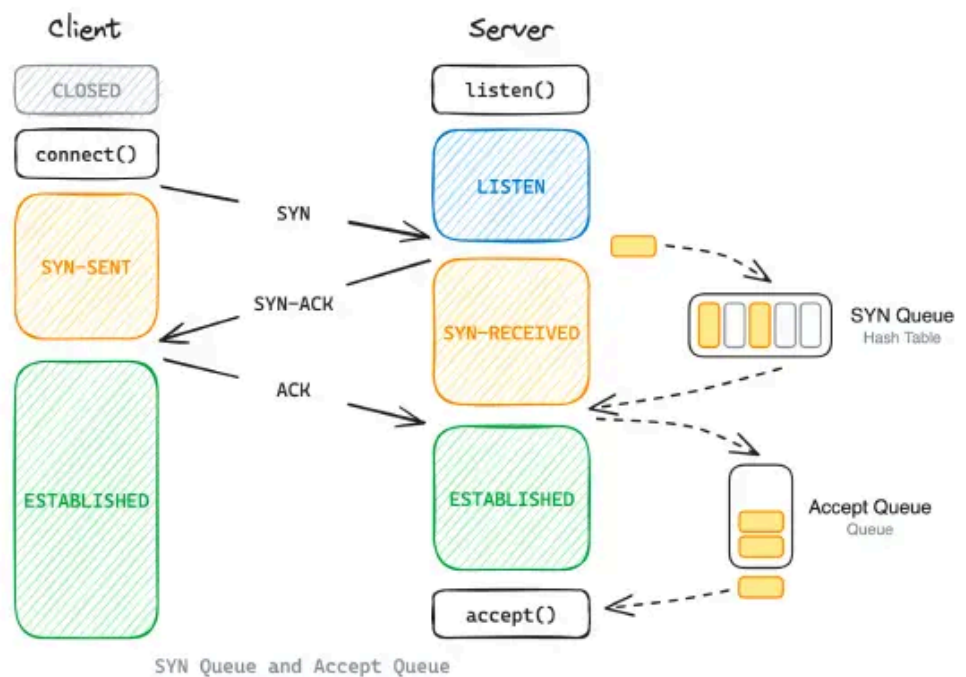
1. The client sends a SYN (synchronize) packet to the server, indicating a desire to establish a connection.

2. The server responds with a SYN-ACK (synchronize-acknowledge) packet, indicating a willingness to establish a connection.
3. The client replies with an ACK (Acknowledge) packet, indicating that it has received the SYN-ACK message and the connection has been established.

Due to network latency, there is always a delay from when the server sends the SYN-ACK packet to when it receives the ACK packet from the client. During this time, the server needs to temporarily store key information about this connection, such as the TCP 4-tuple, MSS (Maximum Segment Size), and window scale factor. Therefore, the Linux kernel maintains a SYN queue to store this connection information.

When the server receives the ACK packet from the client, it removes the connection from the SYN queue and places it in the accept queue, which is also maintained by the kernel, until the application layer calls `accept()` to move the connection from the accept queue.

The main role of the accept queue here is to decouple the network layer and the application layer, allowing the TCP three-way handshake and actual data transmission to proceed in parallel, effectively improving the efficiency of connection request processing.



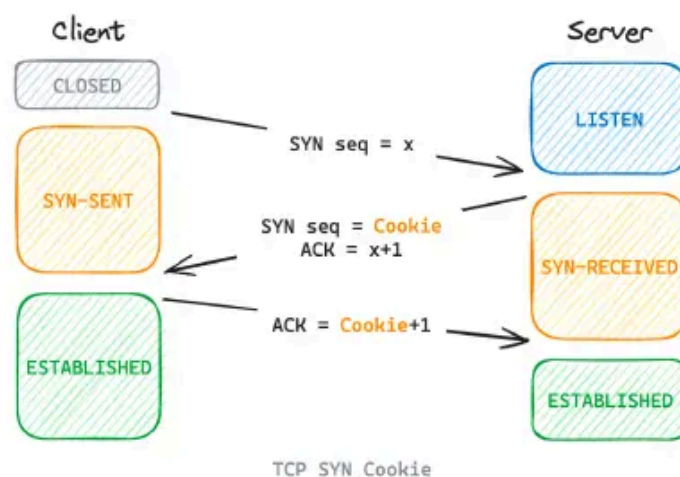
However, the server's resources are limited, and both the SYN and accept queues have maximum length limits. If the client continuously sends SYN packets without responding with the final ACK packet, or if the server's application does not call `accept()` on time, there is a risk of overflow in these two queues.

What Happens When the SYN Queue Overflows?

The server's behavior when the SYN queue overflows is mainly determined by the `net.ipv4.tcp_syncookies` option.

Since the size of the SYN queue is always limited, some attackers may try to attack the server by sending a large number of SYN packets, attempting to exhaust the server's SYN queue to prevent the establishment of legitimate connections. This is commonly known as a SYN flood attack.

The SYN Cookie mechanism was born to solve this problem. Simply put, when this mechanism is enabled, Linux calculates a Cookie based on information such as timestamps and four-tuples when receiving a SYN packet, and then returns it to the client as the sequence number of the SYN-ACK packet. The client returns the sequence number plus one in the ACK, and the server only needs to subtract one to reverse the original Cookie. Therefore, the server no longer needs to put the connection request into the SYN queue.



However, the SYN Cookie mechanism also has some drawbacks:

1. The calculation of the Cookie does not include TCP options such as SACK (Selective Acknowledgment) and Window Scale, and the server will not save these options after the SYN Cookie is enabled, so these features cannot be used. Although starting from Linux kernel v2.6.26, we can enable the TCP Timestamps option (`net.ipv4.tcp_timestamps`), and use the lower 6 bits of the 32-bit timestamp to store these TCP options, TCP Timestamps require the joint support of the client and server to be truly enabled.
2. The Hash calculation used to generate the Cookie increases the load on the server.

Therefore, the `net.ipv4.tcp_syncookies` option currently has three possible values:

- `net.ipv4.tcp_syncookies = 0`, which means to turn off the SYN Cookie mechanism. If the SYN queue is full, the newly arrived SYN message will be discarded.
- `net.ipv4.tcp_syncookies = 1`, which means the SYN Cookie mechanism is only formally enabled when the SYN queue is full.
- `net.ipv4.tcp_syncookies = 2`, which means to unconditionally enable the SYN Cookie mechanism.

In different types and versions of operating systems, the default value of

`net.ipv4.tcp_syncookies` may be different. You can run the following command to check the current value:

```
sysctl -n net.ipv4.tcp_syncookies
```



Considering the potential side effects of SYN Cookies, we generally suggest only enabling SYN Cookies when the SYN queue is full (by setting `net.ipv4.tcp_syncookies` to 1), and prioritize increasing the maximum size of the SYN queue as much as possible. Run the following command to modify this option:

```
sysctl -w net.ipv4.tcp_syncookies=1
```



The Impact of RTT on SYN Queue

The time a connection request stays in the SYN queue is basically the same as the time from when the server sends the SYN-ACK packet to when it receives the ACK packet from the client. In other words, it depends entirely on the Round Trip Time (RTT) of the packet between the client and the server.

The longer the RTT, the easier it is for connection requests to fill up the SYN queue. Suppose the RTT is 200 ms, and the maximum size of the SYN queue is 512, then as long as the number of clients initiating connection requests to the server exceeds 2560 per second, it will cause the SYN queue to overflow.

View the Current SYN Queue Size

Linux does not provide a corresponding kernel parameter to view the current SYN queue size directly. However, previous discussions show that connections in the SYN queue are in the SYN-

RECEIVED status. Therefore, we can indirectly obtain the current SYN queue size by using the `netstat` command to count the number of connections in the SYN-RECEIVED status:

```
sudo netstat -antp | grep SYN_RECV | wc -l
```



How To Confirm the SYN Queue Overflow?

When SYN packets are dropped due to a full SYN queue, the following count in the server will increase accordingly:

```
$ netstat -s | grep "LISTEN"
<Number> SYNs to LISTEN sockets dropped
```



However, SYN packets may also be dropped due to a full accept queue, so it is necessary to make a comprehensive judgment in combination with the situation of the accept queue.

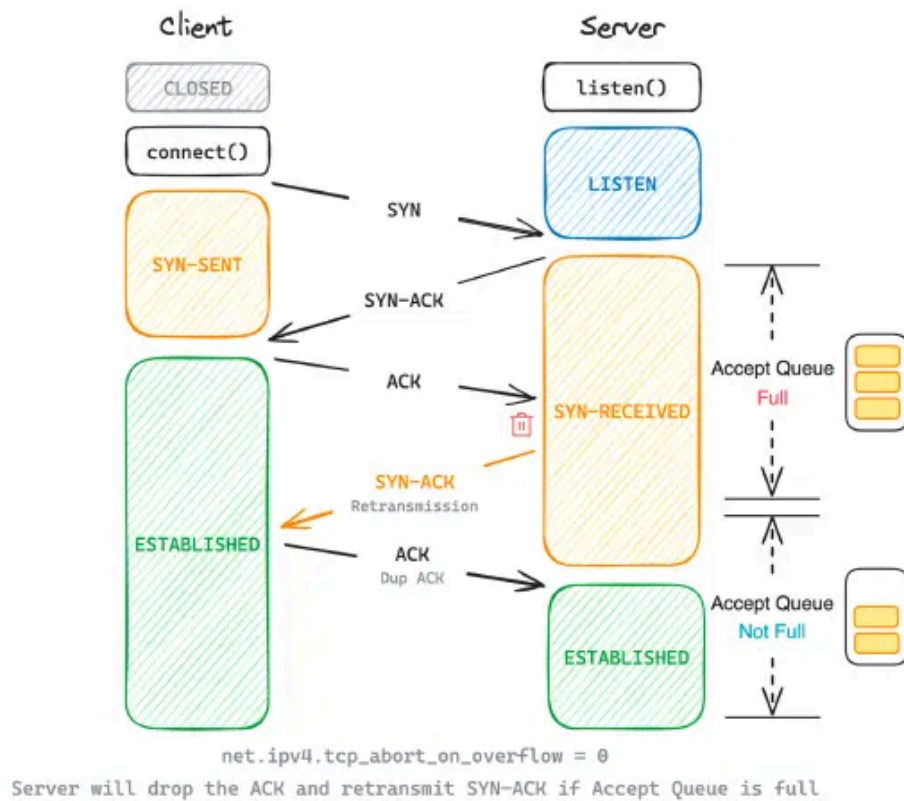
It is worth noting that in CentOS, even if the SYN Cookie mechanism is enabled and the server no longer drops SYN packets, this SYN drop count may still increase:

- When SYN Cookie is set to 1, the newly arrived SYN packets will still increase this count if the SYN queue is full. This can help us evaluate whether we need to increase the maximum size of the SYN queue.
- When SYN Cookie is set to 2, this count has no meaning.

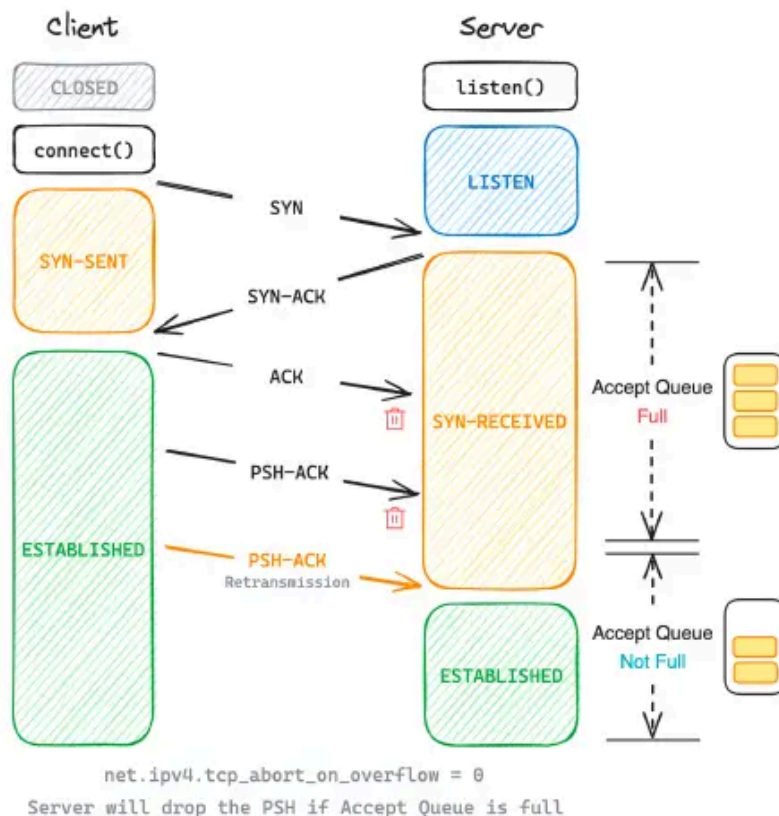
What Happens When the Accept Queue Overflows?

The behavior of the server when the accept queue overflows is mainly determined by the `net.ipv4.tcp_abort_on_overflow` option.

Under normal circumstances, the default value of this option is 0, that is, when the accept queue overflows, the server will directly discard the ACK packet of the third handshake, and regard it as if it has never received this ACK packet, so the server will retransmit the SYN-ACK packet. The maximum number of retransmissions is determined by the `net.ipv4.tcp_synack_retries` option.

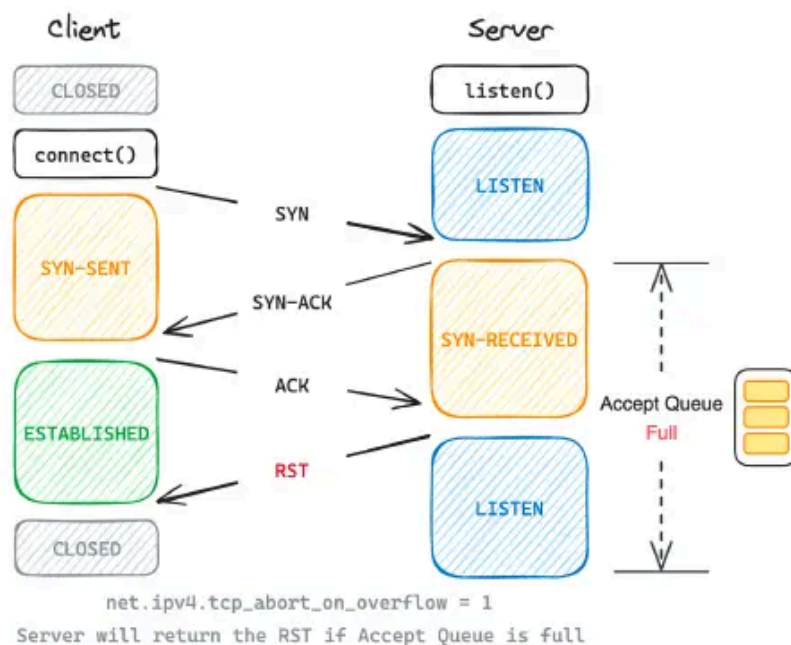


Although the server has discarded the ACK packet, from the client's perspective, the three-way handshake has been completed, so it can send subsequent application data. However, the server will drop these PSH packets carrying application data directly, just like the ACK packets. Since no response is received, the client will continuously retransmit the PSH packets. The maximum number of PSH packet retransmissions is determined by the `net.ipv4.tcp_retries2` option.



The benefit of setting `net.ipv4.tcp_abort_on_overflow` to 0 is that if the SYN-ACK or PSH packets have not reached the maximum number of retransmissions yet, and the upper-level application timely removes the connection to free up space in the accept queue, the connection can directly be restored. This approach is more advantageous for handling burst traffic. However, if the accept queue is too short, causing the client and server to retransmit packets prematurely, it can also waste bandwidth and reduce connection efficiency.

On the other hand, if `net.ipv4.tcp_abort_on_overflow` is set to 1, then the server will directly send an RST packet to the client to close the connection when the accept queue overflows.



So, we generally recommend setting `net.ipv4.tcp_abort_on_overflow` to 0, unless you are certain that the server cannot recover from heavy traffic quickly and wish to notify the client as soon as possible.

Different types and versions of operating systems may behave differently in this regard. The above behavior was validated in CentOS, but Ubuntu seems to adopt a different behavior pattern, and changing `net.ipv4.tcp_abort_on_overflow` does not seem to alter Ubuntu's behavior.

If you're interested, we've provided sample code and steps for simulating accept queue overflow [here](#) so you can experiment in your own environment.

In addition, when the accept queue overflows, even if the SYN Cookie mechanism is enabled, the server will no longer accept new connection requests. This means that the SYN packets reaching the server will be directly dropped, resulting in an increase in the SYN drop count.

View the Current Accept Queue Size

We can use the `ss` command to check the current status of the accept queue. For sockets in the listening state, the second column Recv-Q obtained by the `ss` command represents the current size of the accept queue, and the second column Send-Q represents the maximum size of the accept queue:

```
$ ss -lnt
LISTEN  0      1024    *:1883      *:*
```



How To Confirm the Accept Queue Overflow?

Whenever the server drops a packet due to an overflow of the accept queue, whether it's the SYN packet of the first handshake, the ACK packet of the third handshake, or the PSH packet, the following count in the server will increase by 1:

```
$ netstat -s | grep "overflowed"
<Number> times the listen queue of a socket overflowed
```



Therefore, we can determine whether the accept queue has overflowed by observing whether this count has increased.

Increase the Size of the SYN and Accept Queues

The accept queue is relatively simple. Its maximum size is determined by the backlog parameter in the listen function (for example, `listen(fd, backlog)`) and the Linux kernel parameter `net.core.somaxconn`. Linux always takes the smaller of the `backlog` and `net.core.somaxconn` as the maximum size of the accept queue.

In EMQX, we can set the `backlog` for each listener separately. For the default TCP listener as an example, we just need to add the following configuration in `emqx.conf`:

```
listeners.tcp.default {
  tcp_options {
    backlog = 1024
  }
}
```



If you want to modify the `backlog` of other listeners, you can do so by using the corresponding protocol name and listener name:

```
listeners.[tcp | ssl | ws | wss | quic].<Listener Name> {  
  tcp_options {  
    backlog = 1024  
  }  
}
```



The SYN queue is slightly more complex. Its maximum size is not directly determined by a kernel parameter, but is affected by parameters such as `net.ipv4.tcp_max_syn_backlog` and `net.core.somaxconn`. The effects of these parameters may vary on different operating systems. In CentOS, the maximum size of the SYN queue has the following calculation formula:

```
Max SYN Queue Size = roundup_pow_of_two(  
  max(min(somaxconn, backlog, sysctl_max_syn_backlog), 8) + 1)
```



`roundup_pow_of_two(Num)` means rounding up `Num` to the power of 2. For example, when `Num` is 6, 7, or 8, `roundup_pow_of_two(Num)` always returns 8.

Therefore, if we set `somaxconn` to 64, `tcp_max_syn_backlog` to 128, and the `backlog` of the `listen()` function to 256, then the maximum size of the SYN queue in CentOS will be 256.

In Ubuntu, the size of the SYN queue must be less than the maximum size of the accept queue and less than or equal to 0.75 times `net.ipv4.tcp_max_syn_backlog`. We can convert it to the following formula:

```
Max SYN Queue Size = min(min(somaxconn, backlog), 0.75 * tcp_max_syn_backlog  
+ 1)
```



If we set `somaxconn` to 64, `tcp_max_syn_backlog` to 512, and `backlog` to 256, the maximum size of the accept queue is 64, which is less than 0.75 times `tcp_max_syn_backlog`, so the maximum size of the SYN queue is 64.

If we set `somaxconn` to 1024, `tcp_max_syn_backlog` to 256, and `backlog` to 512, the maximum size of the accept queue is 512, which is greater than `tcp_max_syn_backlog`, so the maximum size of the SYN queue is 193.

Verify the Maximum Size of the SYN Queue

We can verify the maximum size of the SYN queue in the following way:

First, we need a simple TCP server that listens on port 12345 but never calls `accept()` to get connections from the accept queue. Note that you should modify the backlog to your expected value:

```
import socket
import time

def start_server(host, port, backlog):
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    server_socket.bind((host, port))
    server_socket.listen(backlog)

    print(f'Server is listening on {host}:{port}')

    while True:
        time.sleep(3)

if __name__ == '__main__':
    start_server('0.0.0.0', 12345, 256)
```



Then we need to do the following on the server side:

```
# If you are using Ubuntu, you need to disable SYN Cookie
# so that you can see the count change in the next steps
echo 0 > /proc/sys/net/ipv4/tcp_syncookies

# Set somaxconn and tcp_max_syn_backlog to your desired values
echo 64 > /proc/sys/net/core/somaxconn
echo 512 > /proc/sys/net/ipv4/tcp_max_syn_backlog

# Set a 200ms network delay for the server's eth0 interface
# to make the SYN queue more likely to overflow.
#
# When you no longer need this delay, you can run the following
# command to remove it:
#
```



```
# sudo tc qdisc delete dev eth0 root
sudo tc qdisc add dev eth0 root netem delay 200ms
```

After completing the above settings, run the following command to start the TCP server:

```
python3 ./server.py
```

Open another terminal window and run the following command to observe if the SYN queue overflows:

```
watch -n 1 -d 'netstat -s | grep -i "listen"'
```

Install the hping3 tool on the client side:

```
apt-get install hping3
```

Run the following command to send a specified number of SYN packets at a specified rate:

```
# -S, send SYN packet
# -p <Port>, specify the port
# -c <Count>, the number of packets sent
# -i u100, send packets at intervals of 100us
hping3 -S -p 12345 -c 65 -i u100 <Your Hostname>
```

If you are using Ubuntu, and the values of parameters such as `somaxconn` are consistent with the above examples, then after running the `hping3` command, you will see the SYN drop count increase by 1, because the maximum size of the SYN queue is 64 at this time.

Make Changes Permanent

Both `echo 64 > /proc/sys/net/core/somaxconn` and `sysctl -w net.core.somaxconn=64` are temporary changes. Our changes will be invalidated once the user logs out or the system restarts. If we confirm that the current value meets the final expectation, we can write them into `/etc/sysctl.conf`:

```
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 4096
```

Then run `sysctl -p` to make changes take effect immediately and permanently.

Conclusion

In modern operating systems, `net.ipv4.tcp_syncookies` is usually set to 1 by default, and `net.ipv4.tcp_abort_on_overflow` is usually set to 0 by default, so apart from the overflow of the Accept queue causing the server to reject new connection requests, it's hard to directly observe client connection failures.

When the SYN Cookie mechanism is activated, although SYN packets will no longer be dropped, some TCP functions may be limited, and the server load will increase accordingly. Especially in scenarios like IoT and connected vehicles where RTT is higher, the SYN queue is more likely to overflow. Therefore, paying timely attention to the SYN dropped count and adjusting the SYN queue size is necessary.

The accept queue is more likely to overflow when the server is busy. When `tcp_abort_on_overflow` equals 0, a shorter Accept queue may cause the client and server to enter packet retransmission prematurely, increasing network load. If the accept queue continues to overflow, but the server's CPU is not saturated, the accept queue can be appropriately enlarged.

Furthermore, setting the SYN queue and accept queue to a very large value is not good. When the server is busy due to a flood attack or sudden traffic, applying reasonable size limits to these two queues is actually a protection for the server.

In subsequent blogs, we will continue to bring more optimization guides for kernel parameters in Linux systems that affect the performance of EMQX.

EMQX Performance Tuning

♡ Like (1) ✎ Edit 🗉 Feedback

Share



Article By

**EMQX Team**

The EMQX team develops the EMQX Platform, continuously delivering high-performance, scalable MQTT solutions that bridge IoT systems and AI agents for evolving industry needs.

Subscribe to our blogs

[Subscribe](#)

Related Posts

Jan 18, 2024

EMQX Team

EMQX Performance Tuning: Linux Conntrack and MQTT Connections

The connection tracking mechanism in Linux is the foundation of many network applications, it may affect our connection establishment.

Nov 10, 2023

EMQX Team

EMQX Performance Tuning: Max Connections and File Descriptors

While testing the performance of EMQX, if the number of client connections reaches a large number, you may find that EMQX cannot accept more connections. In this article, w...

